TEORIA DO APRENDIZADO ESTATÍSTICO · PROFESSOR RÔMULO

Lógica Fuzzy aplicada ao desempenho de alunos

Sistema de inferência de Mamdani para prever a nota final e apontar alunos em risco de reprovação, com app interativo em Streamlit.

Arthur Almeida e Nicolas Soares

11 de novembro de 2026

1. Introdução

Objetivo. Aplicar a **lógica fuzzy (nebulosa)** à base *Student Performance* (Cortez & Silva, 2008, UCI/Kaggle), com 649 alunos de língua portuguesa de duas escolas de Portugal, para **prever a nota final (G3, de 0 a 20)** e **identificar alunos em risco de reprovação** (G3 < 10), de forma que qualquer professor consiga entender o raciocínio.

O que é lógica fuzzy. Na lógica clássica um valor está dentro ou fora de uma categoria. Na fuzzy, ele tem um **grau de pertinência** de 0 a 1. Quem estuda "entre 2 e 5 horas por semana" não é simplesmente *pouco* ou *muito*: é 37% *pouco* e 63% *médio*, por exemplo. Com isso conseguimos escrever **regras em linguagem comum** (*SE estuda muito E nunca reprovou ENTÃO a nota tende a ser boa*) e obter uma resposta numérica suave, sem saltos bruscos entre categorias.

A base. São 33 colunas originais (hábitos, família, escola e as notas G1, G2 e G3), sem valores ausentes. 100 alunos (15%) reprovaram (G3 < 10) e a nota média é 11.9. O que mais separa as notas:

- **Reprovações anteriores:** média 12.5 para quem nunca reprovou contra 8.6 para quem reprovou uma vez.
- **Ambição:** quem quer cursar ensino superior tem média 12.3, contra 8.8 de quem não quer.
- **Tempo de estudo e álcool no fim de semana:** mais estudo e menos álcool andam com notas maiores.
- **Notas parciais:** G2 tem correlação de 0.92 com G3; sozinha, ela quase resolve o problema.

> **Dois sistemas.** Para mostrar os dois cenários, construímos o **Sistema A (perfil do aluno, sem notas parciais)**, um alerta precoce que funciona antes de qualquer prova, e o **Sistema B (com a nota do 2º período, G2)**, que representa o que o professor já sabe perto do fim do ano.

2. Explicação do código

O projeto tem um **núcleo independente** (`fuzzy_core.py`), onde está toda a lógica fuzzy, e um **app Streamlit** com uma página por etapa. As páginas só chamam o núcleo; assim, o motor pode ser testado sem abrir a interface.

Arquivo	O que faz
`fuzzy_core.py`	Define variáveis, conjuntos fuzzy, regras e o motor de Mamdani; contém os sistemas A e B e as métricas de avaliação.
`principal.py`	Ponto de entrada do app; monta o menu com as 4 páginas.
`pagina1_base.py`	Apresenta a base, explica a ideia e mostra os gráficos que justificam as regras.
`pagina2_conjuntos.py`	Mostra as funções de pertinência de cada variável e a fuzzificação de um valor escolhido.
`pagina3_regras.py`	Simulador: ajusta o perfil de um aluno, mostra regras disparadas, área combinada e nota (centroide).
`pagina4_avaliacao.py`	Compara o fuzzy com chute, regressão linear e floresta aleatória (MAE, R^2, AUC, curva ROC, alerta precoce).
`tests_fuzzy.py`	Confere o motor contra o `scikit-fuzzy` e valida propriedades básicas.
`estilo.py`	Cores, tema dos gráficos e glossário compartilhados.

Como o sistema de Mamdani funciona (4 passos)

1. **Fuzzificação.** Cada entrada vira graus de pertinência por meio de triângulos e trapézios (`Variavel.fuzzificar`). Ex.: tempo de estudo 2 → *pouco* 0,37 e *médio* 0,63.
2. **Avaliação das regras.** Em cada regra *SE … E … ENTÃO …*, o **E** usa o **mínimo** dos graus; o resultado é a **força** da regra (`SistemaFuzzy.forcas`).
3. **Agregação.** O conjunto de saída de cada regra é **cortado** na altura da sua força e todos os cortes são unidos pelo **máximo** (`SistemaFuzzy.agregar`).
4. **Defuzzificação.** O **centroide** (centro de gravidade) da área resultante é a nota prevista (`SistemaFuzzy.prever`).

Decisões de projeto

- **Motor próprio e vetorizado.** Em vez de simular aluno por aluno, o NumPy calcula os 649 de uma vez; o `scikit-fuzzy` fornece as funções de pertinência e serve de referência nos testes.
- **Variáveis escolhidas pela análise dos dados** (página 1): reprovações, ambição de cursar ensino superior, tempo de estudo e álcool no fim de semana para o sistema A; G2, reprovações e estudo para o B. As **faltas ficaram de fora**, pois quase não explicam a nota nesta base.
- **Saída em conjuntos centrados onde as notas reais ficam.** No sistema A são 4 conjuntos (risco ≈ 8, regular ≈ 11,5, boa ≈ 14,5, ótima ≈ 17,5); no B, 5 conjuntos (de 6 a 18), porque G2 é muito informativa e justifica mais resolução.
- **Regras escritas à mão, sem treino automático**, cada uma com uma justificativa em português (tabelas abaixo).
- **Testes.** O motor difere do `scikit-fuzzy` em no máximo 0,014 ponto de nota; as pertinências ficam entre 0 e 1, as notas entre 0 e 20, nenhum aluno fica sem regra ativa, e a nota do sistema B cresce com G2.

Regras do Sistema A (perfil do aluno)

#	Regra	Por quê
1	SE reprovações é algumas ENTÃO nota é risco	Quem já reprovou tem nota média ~8,6, contra ~12,5 de quem nunca reprovou.
2	SE ensino superior é não ENTÃO nota é risco	Sem intenção de ensino superior, a nota média cai para ~8,8 (contra ~12,3).
3	SE reprovações é nenhuma E ensino superior é sim E estudo é pouco ENTÃO nota é regular	Histórico limpo e ambição, mas pouco estudo: nota mediana.
4	SE reprovações é nenhuma E ensino superior é sim E estudo é médio ENTÃO nota é regular	Perfil intermediário em estudo, com histórico limpo e ambição.
5	SE reprovações é nenhuma E ensino superior é sim E estudo é muito ENTÃO nota é boa	Muito estudo, sem reprovação e com ambição: nota boa.
6	SE reprovações é nenhuma E ensino superior é sim E estudo é médio E álcool no fds é baixo ENTÃO nota é boa	Rotina equilibrada (estudo médio e pouco álcool) leva a nota boa.
7	SE reprovações é nenhuma E ensino superior é sim E estudo é muito E álcool no fds é baixo ENTÃO nota é ótima	Combinação ideal: estudo alto, pouco álcool, sem reprovação, com ambição.
8	SE álcool no fds é alto E estudo é pouco ENTÃO nota é risco	Muito álcool no fim de semana com pouco estudo: risco.
9	SE álcool no fds é alto E reprovações é nenhuma E ensino superior é sim ENTÃO nota é regular	Álcool alto segura a nota em regular, mesmo com histórico limpo.

Regras do Sistema B (com nota G2)

#	Regra	Por quê
1	SE nota G2 é muito baixa ENTÃO nota é crítica	Nota muito baixa no 2º período costuma terminar crítica.
2	SE nota G2 é baixa ENTÃO nota é risco	Nota baixa no 2º período tende a continuar baixa.
3	SE nota G2 é média ENTÃO nota é regular	Nota média se mantém média.
4	SE nota G2 é boa ENTÃO nota é boa	Nota boa se mantém boa.
5	SE nota G2 é alta ENTÃO nota é ótima	Nota alta se mantém ótima.
6	SE nota G2 é média E reprovações é algumas ENTÃO nota é risco	Nota média com reprovação prévia tende a cair para a faixa de risco.
7	SE nota G2 é boa E reprovações é algumas ENTÃO nota é regular	Nota boa com reprovação prévia tende a recuar um degrau.
8	SE nota G2 é baixa E estudo é muito ENTÃO nota é regular	Nota baixa, mas com muito estudo: tende a recuperar.
9	SE nota G2 é média E estudo é muito ENTÃO nota é boa	Nota média com muito estudo: tende a subir.

3. Código-fonte comentado

Núcleo fuzzy `fuzzy_core.py`

Variáveis, conjuntos, regras, motor de Mamdani, sistemas A e B e métricas.

```python
"""Núcleo do sistema fuzzy (Mamdani) para prever a nota final (G3) de alunos.

Tudo aqui é independente do Streamlit: as páginas só importam estas funções.
O motor é vetorizado com NumPy (roda os 649 alunos de uma vez) e é conferido,
nos testes, contra o motor do scikit-fuzzy (ver tests_fuzzy.py).

Passo a passo de um sistema Mamdani:
  1. Fuzzificação  -> cada entrada vira graus de pertinência (0 a 1) em cada conjunto.
  2. Regras        -> SE ... E ... ENTÃO ...  (E = mínimo dos graus).
  3. Agregação     -> a saída de cada regra é "cortada" no grau da regra (mínimo)
                      e todas são unidas com o máximo.
  4. Defuzzificação-> o centroide da área resultante vira um número (a nota).
"""
from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path

import numpy as np
import pandas as pd
import skfuzzy as fuzz

DADOS = Path(__file__).parent / "data" / "student-por.csv"

# Nomes curtos das variáveis, usados nas frases das regras.
CURTO = {
    "studytime": "estudo", "failures": "reprovações", "Walc": "álcool no fds",
    "higher_n": "ensino superior", "G2": "nota G2", "G3": "nota final",
}

# Universo da nota final (0 a 20), com passo fino para o centroide ficar suave.
U_NOTA = np.arange(0, 20.01, 0.05)


# ---------------------------------------------------------------------------
# Dados
# ---------------------------------------------------------------------------
def carregar_dados() -> pd.DataFrame:
    """Lê o student-por.csv (separador vírgula; G1 e G2 vêm entre aspas).

    Cria duas colunas auxiliares: `higher_n` (higher yes/no -> 1/0) e
    `reprovado` (1 se G3 < 10, nota de corte do sistema de 0 a 20).
    """
    df = pd.read_csv(DADOS)
    df["higher_n"] = (df["higher"] == "yes").astype(int)
    df["reprovado"] = (df["G3"] < 10).astype(int)
    return df


# ---------------------------------------------------------------------------
# Variáveis e conjuntos fuzzy
# ---------------------------------------------------------------------------
@dataclass
class Variavel:
    nome: str                     # coluna na base (ex.: "studytime")
    rotulo: str                   # nome para mostrar (ex.: "Tempo de estudo")
    universo: tuple[float, float]  # (mínimo, máximo)
    # conjuntos: nome -> ("tri"|"trap", [parâmetros])
    conjuntos: dict[str, tuple[str, list[float]]] = field(default_factory=dict)
    unidade: str = ""

    def x(self, passo: float | None = None) -> np.ndarray:
        lo, hi = self.universo
        passo = passo or (hi - lo) / 400
        return np.arange(lo, hi + passo / 2, passo)

    def pertinencia(self, valores, conjunto: str) -> np.ndarray:
        tipo, p = self.conjuntos[conjunto]
        v = np.asarray(valores, dtype=float)
```

```
        return fuzz.trimf(v, p) if tipo == "tri" else fuzz.trapmf(v, p)

    def fuzzificar(self, valores) -> dict[str, np.ndarray]:
        return {c: self.pertinencia(valores, c) for c in self.conjuntos}


@dataclass
class Regra:
    se: dict[str, str]   # {variável: conjunto}, ligadas por E
    entao: str           # conjunto da saída
    texto: str = ""      # explicação em português

    def __str__(self) -> str:
        cond = " E ".join(f"{v} é {c}" for v, c in self.se.items())
        return f"SE {cond} ENTÃO nota é {self.entao}"

    def descricao(self) -> str:
        """Mesma regra, com nomes curtos e legíveis (para mostrar na tela)."""
        cond = " E ".join(f"{CURTO.get(v, v)} é {c}" for v, c in self.se.items())
        return f"SE {cond} ENTÃO nota é {self.entao}"


@dataclass
class SistemaFuzzy:
    nome: str
    entradas: dict[str, Variavel]
    saida: Variavel
    regras: list[Regra]

    # -- passo 1 e 2: força (grau de ativação) de cada regra, aluno a aluno ----
    def forcas(self, df: pd.DataFrame) -> np.ndarray:
        graus = {n: v.fuzzificar(df[n].to_numpy()) for n, v in self.entradas.items()}
        f = np.empty((len(df), len(self.regras)))
        for j, r in enumerate(self.regras):
            g = np.ones(len(df))
            for var, conj in r.se.items():
                g = np.minimum(g, graus[var][conj])   # E = mínimo
            f[:, j] = g
        return f

    # -- passos 3 e 4: agregação (máximo dos cortes) e centroide ---------------
    def agregar(self, forcas: np.ndarray) -> np.ndarray:
        mf_saida = {c: self.saida.pertinencia(U_NOTA, c) for c in self.saida.conjuntos}
        agg = np.zeros((forcas.shape[0], len(U_NOTA)))
        for j, r in enumerate(self.regras):
            corte = np.minimum(forcas[:, [j]], mf_saida[r.entao][None, :])
            agg = np.maximum(agg, corte)
        return agg

    def prever(self, df: pd.DataFrame, valor_sem_regra: float | None = None) -> np.ndarray:
        agg = self.agregar(self.forcas(df))
        area = agg.sum(axis=1)
        centro = (agg * U_NOTA[None, :]).sum(axis=1) / np.where(area == 0, 1, area)
        base = valor_sem_regra if valor_sem_regra is not None else float(U_NOTA.mean())
        return np.where(area == 0, base, centro)

    def prever_um(self, **entradas: float) -> dict:
        """Versão para o simulador: devolve tudo que a tela precisa desenhar."""
        df = pd.DataFrame([entradas])
        f = self.forcas(df)[0]
        agg = self.agregar(f[None, :])[0]
        area = agg.sum()
        nota = float((agg * U_NOTA).sum() / area) if area > 0 else float("nan")
        return {"nota": nota, "forcas": f, "agregado": agg}


# ---------------------------------------------------------------------------
# Saída comum aos dois sistemas: nota final de 0 a 20
# Partição padrão: 4 triângulos espaçados em ~3 pontos, centrados onde as notas
# reais costumam ficar (risco ~8, regular ~11,5, boa ~14,5, ótima ~17,5).
# ---------------------------------------------------------------------------
def _saida() -> Variavel:
    return Variavel(
        "G3", "Nota final (G3)", (0, 20),
        {
            "risco":   ("tri",  [5, 8, 11]),
            "regular": ("tri",  [9, 11.5, 14]),
            "boa":     ("tri",  [12, 14.5, 17]),
            "ótima":   ("trap", [15, 17.5, 20, 20]),
        },
        "pontos",
```

```
    )


def _particao5() -> dict[str, tuple[str, list[float]]]:
    """5 conjuntos centrados em 6, 9, 12, 15 e 18 (usada no sistema B, onde G2
    já é muito informativa e uma partição mais fina deixa a saída mais precisa)."""
    return {
        "crítica": ("trap", [0, 0, 6, 9]),
        "risco":   ("tri",  [6, 9, 12]),
        "regular": ("tri",  [9, 12, 15]),
        "boa":     ("tri",  [12, 15, 18]),
        "ótima":   ("trap", [15, 18, 20, 20]),
    }


# ---------------------------------------------------------------------------
# Sistema A — perfil do aluno (SEM notas parciais): hábitos e histórico
# Serve como "alerta precoce": dá para usar antes de qualquer prova.
# ---------------------------------------------------------------------------
def sistema_perfil() -> SistemaFuzzy:
    ent = {
        "studytime": Variavel("studytime", "Tempo de estudo semanal", (1, 4), {
            "pouco": ("trap", [1, 1, 1.5, 2.3]),
            "médio": ("tri",  [1.5, 2.3, 3.2]),
            "muito": ("trap", [2.5, 3.3, 4, 4]),
        }, "1 = <2h · 2 = 2–5h · 3 = 5–10h · 4 = >10h"),
        "failures": Variavel("failures", "Reprovações anteriores", (0, 3), {
            "nenhuma": ("trap", [0, 0, 0.2, 1]),
            "algumas": ("trap", [0.3, 1, 3, 3]),
        }, "quantidade"),
        "Walc": Variavel("Walc", "Álcool no fim de semana", (1, 5), {
            "baixo": ("trap", [1, 1, 1.5, 3]),
            "alto":  ("trap", [2.5, 4, 5, 5]),
        }, "1 = muito baixo … 5 = muito alto"),
        "higher_n": Variavel("higher_n", "Quer cursar ensino superior", (0, 1), {
            "não": ("trap", [0, 0, 0.1, 0.6]),
            "sim": ("trap", [0.4, 0.9, 1, 1]),
        }, "0 = não · 1 = sim"),
    }
    R = Regra
    regras = [
        R({"failures": "algumas"}, "risco",
          "Quem já reprovou tem nota média ~8,6, contra ~12,5 de quem nunca reprovou."),
        R({"higher_n": "não"}, "risco",
          "Sem intenção de ensino superior, a nota média cai para ~8,8 (contra ~12,3)."),
        R({"failures": "nenhuma", "higher_n": "sim", "studytime": "pouco"}, "regular",
          "Histórico limpo e ambição, mas pouco estudo: nota mediana."),
        R({"failures": "nenhuma", "higher_n": "sim", "studytime": "médio"}, "regular",
          "Perfil intermediário em estudo, com histórico limpo e ambição."),
        R({"failures": "nenhuma", "higher_n": "sim", "studytime": "muito"}, "boa",
          "Muito estudo, sem reprovação e com ambição: nota boa."),
        R({"failures": "nenhuma", "higher_n": "sim", "studytime": "médio", "Walc": "baixo"}, "boa",
          "Rotina equilibrada (estudo médio e pouco álcool) leva a nota boa."),
        R({"failures": "nenhuma", "higher_n": "sim", "studytime": "muito", "Walc": "baixo"}, "ótima",
          "Combinação ideal: estudo alto, pouco álcool, sem reprovação, com ambição."),
        R({"Walc": "alto", "studytime": "pouco"}, "risco",
          "Muito álcool no fim de semana com pouco estudo: risco."),
        R({"Walc": "alto", "failures": "nenhuma", "higher_n": "sim"}, "regular",
          "Álcool alto segura a nota em regular, mesmo com histórico limpo."),
    ]
    return SistemaFuzzy("A · Perfil do aluno (sem notas parciais)", ent, _saida(), regras)


# ---------------------------------------------------------------------------
# Sistema B — com a nota do 2º período (G2): o que o professor já sabe antes do fim
# ---------------------------------------------------------------------------
def sistema_com_g2() -> SistemaFuzzy:
    p5 = _particao5()
    entrada_g2 = {
        "muito baixa": p5["crítica"], "baixa": p5["risco"], "média": p5["regular"],
        "boa": p5["boa"], "alta": p5["ótima"],
    }
    ent = {
        "G2": Variavel("G2", "Nota do 2º período (G2)", (0, 20), entrada_g2, "pontos"),
        "failures": Variavel("failures", "Reprovações anteriores", (0, 3), {
            "nenhuma": ("trap", [0, 0, 0.2, 1]),
            "algumas": ("trap", [0.3, 1, 3, 3]),
        }, "quantidade"),
        "studytime": Variavel("studytime", "Tempo de estudo semanal", (1, 4), {
            "pouco": ("trap", [1, 1, 1.5, 2.3]),
            "muito": ("trap", [2.5, 3.3, 4, 4]),
```

```
        }, "1 = <2h · 2 = 2–5h · 3 = 5–10h · 4 = >10h"),
    }
    saida = Variavel("G3", "Nota final (G3)", (0, 20), p5, "pontos")
    R = Regra
    regras = [
        R({"G2": "muito baixa"}, "crítica", "Nota muito baixa no 2º período costuma terminar crítica."),
        R({"G2": "baixa"}, "risco", "Nota baixa no 2º período tende a continuar baixa."),
        R({"G2": "média"}, "regular", "Nota média se mantém média."),
        R({"G2": "boa"}, "boa", "Nota boa se mantém boa."),
        R({"G2": "alta"}, "ótima", "Nota alta se mantém ótima."),
        R({"G2": "média", "failures": "algumas"}, "risco",
          "Nota média com reprovação prévia tende a cair para a faixa de risco."),
        R({"G2": "boa", "failures": "algumas"}, "regular",
          "Nota boa com reprovação prévia tende a recuar um degrau."),
        R({"G2": "baixa", "studytime": "muito"}, "regular",
          "Nota baixa, mas com muito estudo: tende a recuperar."),
        R({"G2": "média", "studytime": "muito"}, "boa",
          "Nota média com muito estudo: tende a subir."),
    ]
    return SistemaFuzzy("B · Com a nota do 2º período (G2)", ent, saida, regras)


SISTEMAS = {"A": sistema_perfil, "B": sistema_com_g2}


# ---------------------------------------------------------------------------
# Avaliação
# ---------------------------------------------------------------------------
def metricas(y_real, y_prev, corte: float = 10.0) -> dict:
    y_real = np.asarray(y_real, float)
    y_prev = np.asarray(y_prev, float)
    erro = y_prev - y_real
    aprov_real = y_real >= corte
    aprov_prev = y_prev >= corte
    return {
        "MAE": float(np.mean(np.abs(erro))),
        "RMSE": float(np.sqrt(np.mean(erro ** 2))),
        "R²": float(1 - np.sum(erro ** 2) / np.sum((y_real - y_real.mean()) ** 2)),
        "Acerto aprovado/reprovado": float(np.mean(aprov_real == aprov_prev)),
    }
```

Entrada do app `principal.py`

Registra as quatro páginas na navegação do Streamlit.

```
"""Ponto de entrada do app.  Rode com:  streamlit run principal.py"""
import streamlit as st

st.set_page_config(page_title="Lógica Fuzzy · Desempenho de Alunos", page_icon="🎓", layout="wide")

paginas = [
    st.Page("pagina1_base.py", title="1 · A base de dados", icon="📊"),
    st.Page("pagina2_conjuntos.py", title="2 · Conjuntos fuzzy", icon="📈"),
    st.Page("pagina3_regras.py", title="3 · Regras e simulador", icon="🎛️"),
    st.Page("pagina4_avaliacao.py", title="4 · Resultados", icon="✅"),
]
st.navigation(paginas).run()

st.sidebar.markdown("---")
st.sidebar.caption(
    "Base: *Student Performance* (Cortez & Silva, 2008) · UCI / Kaggle — português, 649 alunos."
)
```

Estilo e glossário `estilo.py`

Paleta, tema dos gráficos e glossário usado nas páginas.

```
"""Estilo visual e funções de apoio compartilhadas pelas páginas do app."""
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import streamlit as st

TINTA = "#1F2937"
```

```
CINZA = "#9CA3AF"
GRADE = "#E5E7EB"
AZUL = "#2563EB"
VERDE = "#059669"
LARANJA = "#D97706"
VERMELHO = "#DC2626"
ROXO = "#7C3AED"
# uma cor por conjunto fuzzy, na ordem natural de "pior" para "melhor"
CORES_CONJUNTOS = [VERMELHO, LARANJA, AZUL, VERDE, ROXO]

plt.rcParams.update({
    "figure.dpi": 130,
    "font.size": 10,
    "axes.edgecolor": CINZA,
    "axes.labelcolor": TINTA,
    "axes.titleweight": "bold",
    "axes.titlesize": 12,
    "axes.titlecolor": TINTA,
    "xtick.color": TINTA,
    "ytick.color": TINTA,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.color": GRADE,
    "grid.linewidth": 0.8,
    "axes.axisbelow": True,
    "legend.frameon": False,
})


def como_ler(texto: str) -> None:
    """Legenda curta abaixo de cada gráfico explicando como interpretá-lo."""
    st.caption(f"**Como ler:** {texto}")


def configurar_pagina(titulo: str) -> None:
    st.title(titulo)


def glossario() -> None:
    with st.expander("📖 Glossário rápido (clique para abrir)"):
        st.markdown(
            """
- **Lógica fuzzy (nebulosa):** em vez de dizer só "sim" ou "não", diz **o quanto** algo é verdade (de 0 a 1).
  Ex.: 3 faltas são "poucas faltas" com grau 0,9, e "faltas médias" com grau 0,1.
- **Conjunto fuzzy:** uma categoria com fronteira suave, como *poucas faltas*, *estuda muito*, *nota boa*.
- **Função de pertinência:** a curva que diz o grau (0 a 1) de cada valor em cada conjunto.
- **Fuzzificação:** transformar o número medido (ex.: 6 faltas) em graus de pertinência.
- **Regra fuzzy:** frase do tipo *SE estuda muito E nunca reprovou ENTÃO nota é boa*.
  O **E** vale o menor dos graus (o "elo mais fraco").
- **Inferência de Mamdani:** cada regra "corta" o conjunto de saída na altura do seu grau; juntam-se todos os
cortes.
- **Defuzzificação (centroide):** o "centro de gravidade" da área resultante vira **um número**: a nota prevista.
- **MAE:** erro médio absoluto, em pontos de nota (quanto menor, melhor).
- **AUC:** de 0,5 (chute) a 1 (perfeito); mede se o modelo coloca os alunos em risco acima dos demais.
"""
        )
```

Página 1 · A base de dados `pagina1_base.py`

Distribuição das notas e gráficos de médias por grupo.

```
"""Página 1 — A base de dados e a ideia da lógica fuzzy."""
import matplotlib.pyplot as plt
import numpy as np
import streamlit as st

import estilo as e
from fuzzy_core import carregar_dados

e.configurar_pagina("📊 Lógica Fuzzy aplicada ao desempenho de alunos")


@st.cache_data
def dados():
    return carregar_dados()
```

```
df = dados()
n = len(df)

st.markdown(
    """
**Objetivo:** prever a **nota final (G3, de 0 a 20)** de alunos do ensino médio português usando **lógica fuzzy**,
com regras que um professor entenderia e poderia conferir, sem "caixa-preta".

**Por que fuzzy?** Hábitos de estudo não têm fronteiras nítidas. Quem estuda "entre 2 e 5 horas" não é
*pouco* nem *muito*, é um pouco dos dois. A lógica fuzzy trabalha com esses **graus** (0 a 1) e permite
escrever regras em linguagem comum, como *"SE estuda muito E nunca reprovou ENTÃO a nota tende a ser boa"*.
"""
)
e.glossario()

c1, c2, c3, c4 = st.columns(4)
c1.metric("Alunos", n)
c2.metric("Nota final média", f"{df.G3.mean():.1f}")
c3.metric("Reprovados (G3 < 10)", f"{df.reprovado.sum()} ({df.reprovado.mean():.0%})")
c4.metric("Valores ausentes", int(df.isna().sum().sum()))

st.subheader("Como as notas se distribuem")
fig, ax = plt.subplots(figsize=(8, 3.4))
cont = df.G3.value_counts().sort_index()
cores = [e.VERMELHO if k < 10 else e.AZUL for k in cont.index]
ax.bar(cont.index, cont.values, color=cores, width=0.8)
ax.axvline(9.5, color=e.TINTA, ls="--", lw=1)
ax.text(9.3, cont.max() * 0.95, "nota de corte (10)", ha="right", va="top", fontsize=9, color=e.TINTA)
ax.set_xlabel("Nota final (G3)")
ax.set_ylabel("Número de alunos")
ax.set_xticks(range(0, 21, 2))
fig.tight_layout()
st.pyplot(fig)
e.como_ler(
    "cada barra é uma nota; as vermelhas são reprovações (abaixo de 10). A maioria fica entre 10 e 14, "
    f"e há {int((df.G3 == 0).sum())} alunos com nota zero (provavelmente desistências), uma cauda difícil de
prever."
)

st.subheader("O que muda a nota? (antes de qualquer modelo)")
fig, axs = plt.subplots(2, 2, figsize=(9, 6))


def barras(ax, serie, rotulos, titulo, xlabel):
    g = df.groupby(serie).G3.agg(["mean", "count"])
    cores = [e.AZUL] * len(g)
    ax.bar(range(len(g)), g["mean"], color=cores)
    for i, (m, c) in enumerate(zip(g["mean"], g["count"])):
        ax.text(i, m + 0.2, f"{m:.1f}", ha="center", fontsize=9, fontweight="bold", color=e.TINTA)
        ax.text(i, 0.4, f"n={c}", ha="center", fontsize=8, color="white")
    ax.set_xticks(range(len(g)))
    ax.set_xticklabels(rotulos)
    ax.set_ylim(0, 16)
    ax.set_title(titulo)
    ax.set_xlabel(xlabel)
    ax.set_ylabel("Nota final média")


barras(axs[0, 0], "studytime", ["<2h", "2–5h", "5–10h", ">10h"], "Tempo de estudo", "horas por semana")
barras(axs[0, 1], "failures", ["0", "1", "2", "3"], "Reprovações anteriores", "reprovações")
barras(axs[1, 0], "higher", ["não", "sim"], "Quer cursar ensino superior?", "")
barras(axs[1, 1], "Walc", ["1", "2", "3", "4", "5"], "Álcool no fim de semana", "1 = muito baixo … 5 = muito
alto")
fig.tight_layout()
st.pyplot(fig)
e.como_ler(
    "cada barra é a nota média de um grupo (o *n* mostra quantos alunos há nele). "
    "Quem já reprovou e quem não pretende fazer faculdade tem nota bem menor; quem estuda mais e bebe menos tende
a ir melhor."
)

g_f = df.groupby("failures").G3.mean()
g_h = df.groupby("higher").G3.mean()
g_s = df.groupby("studytime").G3.mean()
corr = df[["G1", "G2", "G3"]].corr()
st.subheader("Principais achados")
st.markdown(
    f"""
- **Reprovações pesam muito:** quem nunca reprovou tem média **{g_f[0]:.1f}**; quem reprovou uma vez, **
{g_f[1]:.1f}**.
- **Ambição importa:** quem quer ensino superior tem média **{g_h['yes']:.1f}**, contra **{g_h['no']:.1f}** de
```

```
quem não quer.
- **Estudo ajuda, com limite:** a média sobe de **{g_s[1]:.1f}** (<2h) para **{g_s[3]:.1f}** (5–10h) e estabiliza
depois.
- **As notas parciais dominam:** a correlação de G2 com a nota final é **{corr.loc['G2','G3']:.2f}** (G1:
{corr.loc['G1','G3']:.2f}).
  Por isso montamos **dois sistemas**: o **A** usa só o perfil do aluno (serve como alerta precoce, antes de
qualquer prova)
  e o **B** acrescenta a nota do 2º período.
- **Faltas quase não explicam a nota** nesta base (correlação de {df[['absences','G3']].corr().iloc[0,1]:.2f}),
por isso ficaram fora das regras.
"""
)

with st.expander("Ver uma amostra dos dados"):
    st.dataframe(
        df[["school", "sex", "age", "studytime", "failures", "higher", "Walc", "absences", "G1", "G2",
"G3"]].head(15),
        width="stretch",
    )
```

Página 2 · Conjuntos fuzzy `pagina2_conjuntos.py`

Funções de pertinência e fuzzificação interativa.

```
"""Página 2 — Conjuntos fuzzy e fuzzificação."""
import matplotlib.pyplot as plt
import numpy as np
import streamlit as st

import estilo as e
from fuzzy_core import SISTEMAS

e.configurar_pagina("📈 Conjuntos fuzzy: do número ao grau")

st.markdown(
    """
Aqui cada variável vira **conjuntos fuzzy**. Em vez de "estuda pouco: sim/não", cada valor recebe
um **grau de pertinência** de 0 a 1 em cada conjunto. As curvas abaixo (**funções de pertinência**) são
triângulos e trapézios, desenhados a partir do que os dados mostram (por exemplo, onde a nota média muda de
faixa).
**Mova o controle** para ver a fuzzificação de um valor.
"""
)

escolha = st.radio(
    "Sistema",
    list(SISTEMAS),
    format_func=lambda k: SISTEMAS[k]().nome,
    horizontal=True,
)
sis = SISTEMAS[escolha]()

todas = {**sis.entradas, "G3": sis.saida}
for nome, var in todas.items():
    st.markdown("---")
    lo, hi = var.universo
    eh_saida = nome == "G3"
    st.subheader(("🎯 Saída: " if eh_saida else "") + var.rotulo)
    if var.unidade:
        st.caption(f"Escala: {var.unidade}")
    passo = 1 if nome in ("studytime", "failures", "Walc") else (1 if nome == "higher_n" else 0.5)
    inteiro = nome in ("studytime", "failures", "Walc", "higher_n")
    padrao = (lo + hi) / 2
    if inteiro:
        padrao = int(round(padrao))
    if nome == "G3":
        valor = st.slider("Valor para fuzzificar", float(lo), float(hi), 12.0, 0.5, key=f"s_{escolha}_{nome}")
    elif inteiro:
        valor = st.slider("Valor para fuzzificar", int(lo), int(hi), int(padrao), 1, key=f"s_{escolha}_{nome}")
    else:
        valor = st.slider("Valor para fuzzificar", float(lo), float(hi), float(padrao), float(passo),
key=f"s_{escolha}_{nome}")

    x = np.linspace(lo, hi, 600)
    fig, ax = plt.subplots(figsize=(8, 2.8))
    graus = {}
    for i, c in enumerate(var.conjuntos):
        cor = e.CORES_CONJUNTOS[i % len(e.CORES_CONJUNTOS)]
```

```
        mu = var.pertinencia(x, c)
        ax.plot(x, mu, color=cor, lw=2.2, label=c)
        ax.fill_between(x, mu, alpha=0.12, color=cor)
        g = float(var.pertinencia([valor], c)[0])
        graus[c] = g
        if g > 0:
            ax.plot([valor], [g], "o", color=cor, ms=8, zorder=5)
            ax.plot([lo, valor], [g, g], ls=":", color=cor, lw=1)
    ax.axvline(valor, color=e.TINTA, ls="--", lw=1)
    ax.set_ylim(-0.03, 1.1)
    ax.set_xlim(lo, hi)
    ax.set_xlabel(var.rotulo)
    ax.set_ylabel("Grau de pertinência")
    ax.legend(loc="lower center", bbox_to_anchor=(0.5, 1.0), ncol=len(var.conjuntos), fontsize=8)
    fig.tight_layout()
    st.pyplot(fig)

    ativos = [f"**{c}** = {g:.2f}" for c, g in graus.items() if g > 0]
    st.markdown(f"Valor **{valor:g}** pertence a: " + (" · ".join(ativos) if ativos else "nenhum conjunto"))
    e.como_ler(
        "cada curva é um conjunto; a altura mostra o grau (1 = pertence totalmente, 0 = não pertence). "
        "A linha tracejada é o valor escolhido e os pontos mostram os graus que ele recebe. "
        "Onde as curvas se cruzam, o valor é um pouco de cada conjunto, e é aí que a lógica fuzzy se diferencia da
lógica clássica."
    )
```

Página 3 · Regras e simulador `pagina3_regras.py`

Regras disparadas, agregação e centroide de um aluno.

```
"""Página 3 — Regras fuzzy e simulador interativo."""
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import streamlit as st

import estilo as e
from fuzzy_core import SISTEMAS, U_NOTA

e.configurar_pagina("🎛️ Regras fuzzy e simulador")

st.markdown(
    """
Aqui está o "cérebro" do sistema: as **regras SE–ENTÃO**. Ajuste o perfil de um aluno nos controles e veja,
passo a passo, **quais regras disparam**, como as respostas são **combinadas** e qual **nota** sai no final.
"""
)

escolha = st.radio("Sistema", list(SISTEMAS), format_func=lambda k: SISTEMAS[k]().nome, horizontal=True)
sis = SISTEMAS[escolha]()

st.sidebar.header("Perfil do aluno")
entradas = {}
for nome, var in sis.entradas.items():
    lo, hi = var.universo
    if nome == "higher_n":
        entradas[nome] = 1 if st.sidebar.selectbox(var.rotulo, ["sim", "não"], key=f"p_{escolha}_{nome}") == "sim"
else 0
    elif nome in ("studytime", "failures", "Walc"):
        padrao = {"studytime": 2, "failures": 0, "Walc": 2}[nome]
        entradas[nome] = st.sidebar.slider(var.rotulo, int(lo), int(hi), padrao, 1, help=var.unidade,
key=f"p_{escolha}_{nome}")
    else:
        entradas[nome] = st.sidebar.slider(var.rotulo, float(lo), float(hi), 11.0, 0.5, help=var.unidade,
key=f"p_{escolha}_{nome}")

res = sis.prever_um(**entradas)
forcas, agg, nota = res["forcas"], res["agregado"], res["nota"]

c1, c2, c3 = st.columns(3)
c1.metric("Nota prevista (G3)", f"{nota:.1f}")
c2.metric("Situação", "Aprovado" if nota >= 10 else "Em risco de reprovação")
c3.metric("Regras ativas", f"{int((forcas > 0.01).sum())} de {len(sis.regras)}")

st.subheader("1 · Quais regras dispararam?")
linhas_html = []
for r, f in zip(sis.regras, forcas):
    cor = e.AZUL if f > 0.01 else e.GRADE
```

```
    opac = "1" if f > 0.01 else "0.55"
    linhas_html.append(
        f"<tr style='opacity:{opac};border-bottom:1px solid #F3F4F6'>"
        f"<td style='padding:6px 8px'><b>{r.descricao()}</b><br><span style='color:#6B7280;font-size:0.85em'>
{r.texto}</span></td>"
        f"<td style='padding:6px 8px;width:150px'>"
        f"<div style='background:{e.GRADE};border-radius:4px;height:12px'><div style='background:{cor};width:
{f*100:.0f}%;height:12px;border-radius:4px'></div></div>"
        f"<div style='text-align:right;font-size:0.85em'>{f:.2f}</div></td></tr>"
    )
st.markdown(
    "<table style='width:100%;border-collapse:collapse'>"
    "<tr style='border-bottom:2px solid #E5E7EB;text-align:left'><th style='padding:6px 8px'>Regra</th><th
style='padding:6px 8px'>Força (0 a 1)</th></tr>"
    + "".join(linhas_html)
    + "</table>",
    unsafe_allow_html=True,
)
e.como_ler(
    "a **força** de cada regra é o menor grau entre as condições (o **E** vale o elo mais fraco). "
    "Força 0 = a regra não se aplica a este aluno; quanto mais perto de 1, mais ela pesa na decisão."
)

st.subheader("2 · Como as regras viram uma nota")
nomes_saida = list(sis.saida.conjuntos)
cores = {c: e.CORES_CONJUNTOS[i % len(e.CORES_CONJUNTOS)] for i, c in enumerate(nomes_saida)}
fig, ax = plt.subplots(figsize=(9, 3.6))
for c in nomes_saida:
    ax.plot(U_NOTA, sis.saida.pertinencia(U_NOTA, c), color=cores[c], lw=1.2, ls="--", alpha=0.7)
ax.fill_between(U_NOTA, agg, color=e.AZUL, alpha=0.35, label="área resultante (regras combinadas)")
ax.plot(U_NOTA, agg, color=e.AZUL, lw=2)
if agg.sum() > 0:
    ax.axvline(nota, color=e.TINTA, lw=2)
    ax.text(nota + 0.2, max(agg.max(), 0.2) * 1.02, f"centroide = {nota:.1f}", fontsize=10, fontweight="bold",
color=e.TINTA)
ax.axvline(9.5, color=e.CINZA, ls=":", lw=1)
ax.set_xlim(0, 20)
ax.set_ylim(0, 1.15)
ax.set_xlabel("Nota final (G3)")
ax.set_ylabel("Grau")
for c in nomes_saida:
    cx = U_NOTA[np.argmax(sis.saida.pertinencia(U_NOTA, c))]
    ax.text(cx, 1.07, c, color=cores[c], ha="center", fontsize=9, fontweight="bold")
fig.tight_layout()
st.pyplot(fig)
e.como_ler(
    "as linhas tracejadas são os conjuntos de saída (de risco a ótima). Cada regra ativa 'corta' o seu conjunto na
altura da sua força; "
    "a área azul é a união desses cortes. A **linha preta** é o **centroide** (centro de gravidade da área): é a
nota prevista. "
    "A linha pontilhada cinza marca o corte de aprovação (10)."
)

with st.expander("Como o cálculo é feito (em 4 passos)"):
    st.markdown(
        """
1. **Fuzzificação:** cada entrada vira graus de pertinência (página 2).
2. **Avaliação das regras:** em cada regra, o **E** usa o **mínimo** dos graus → *força* da regra.
3. **Agregação:** o conjunto de saída de cada regra é cortado na força da regra; todos os cortes são unidos pelo
**máximo**.
4. **Defuzzificação:** o **centroide** da área resultante dá a nota.

O código está em `fuzzy_core.py` (motor próprio, vetorizado) e é conferido contra o `scikit-fuzzy` em
`tests_fuzzy.py`.
"""
    )
```

Página 4 · Resultados `pagina4_avaliacao.py`

Comparação com chute, regressão linear e floresta aleatória.

```
"""Página 4 — Resultados: o sistema fuzzy contra a base real e contra modelos tradicionais."""
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import streamlit as st
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
```

```python
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.model_selection import KFold, cross_val_predict

import estilo as e
from fuzzy_core import SISTEMAS, carregar_dados, metricas

e.configurar_pagina("✅ Resultados: o fuzzy acerta?")

st.markdown(
    """
*Comparamos os dois sistemas fuzzy com **três referências**: um chute (sempre prever a média), uma **regressão linear** e uma
**floresta aleatória** (as duas com validação cruzada de 5 partes, para ser justo). As regras fuzzy foram escritas à mão a partir
do que a página 1 mostrou; **não há treino automático**.*
"""
)

FEATURES = {"A": ["studytime", "failures", "Walc", "higher_n"], "B": ["G2", "failures", "studytime"]}


@st.cache_data(show_spinner="Calculando previsões...")
def calcular():
    df = carregar_dados()
    cv = KFold(5, shuffle=True, random_state=42)
    saida = {}
    for k, f in SISTEMAS.items():
        sis = f()
        prev = {"Fuzzy": sis.prever(df), "Chute (média)": np.full(len(df), df.G3.mean())}
        prev["Regressão linear"] = cross_val_predict(LinearRegression(), df[FEATURES[k]], df.G3, cv=cv)
        prev["Floresta aleatória"] = cross_val_predict(
            RandomForestRegressor(300, min_samples_leaf=5, random_state=0), df[FEATURES[k]], df.G3, cv=cv
        )
        saida[k] = prev
    return df, saida


df, prev = calcular()

escolha = st.radio("Sistema", list(SISTEMAS), format_func=lambda k: SISTEMAS[k]().nome, horizontal=True)
p = prev[escolha]

linhas = []
for nome, y in p.items():
    m = metricas(df.G3, y)
    auc = roc_auc_score(df.reprovado, -y) if nome != "Chute (média)" else 0.5
    linhas.append({"Modelo": nome, "Erro médio (MAE)": m["MAE"], "R²": m["R²"], "AUC (risco de reprovar)": auc})
tab = pd.DataFrame(linhas)

m_f = tab[tab.Modelo == "Fuzzy"].iloc[0]
m_l = tab[tab.Modelo == "Regressão linear"].iloc[0]
a, b, c = st.columns(3)
a.metric("Erro médio do fuzzy", f"{m_f['Erro médio (MAE)']:.2f} pts", f"{m_f['Erro médio (MAE)'] - m_l['Erro médio (MAE)']:+.2f} vs regressão", delta_color="inverse")
b.metric("AUC do fuzzy", f"{m_f['AUC (risco de reprovar)']:.2f}", f"{m_f['AUC (risco de reprovar)'] - m_l['AUC (risco de reprovar)']:+.2f} vs regressão")
c.metric("R² do fuzzy", f"{m_f['R²']:.2f}")

st.dataframe(
    tab.style.format({"Erro médio (MAE)": "{:.2f}", "R²": "{:.2f}", "AUC (risco de reprovar)": "{:.2f}"}),
    hide_index=True,
    width="stretch",
)
e.como_ler(
    "**MAE** = quantos pontos de nota o modelo erra em média (menor é melhor). **AUC** mede se o modelo põe os alunos que "
    "realmente reprovaram acima dos demais na lista de risco (0,5 = chute, 1 = perfeito). **R²** = fração da variação das notas explicada."
)

col1, col2 = st.columns(2)
with col1:
    st.subheader("Real × previsto")
    fig, ax = plt.subplots(figsize=(4.6, 4.4))
    jit = np.random.default_rng(0).uniform(-0.25, 0.25, len(df))
    ax.scatter(df.G3 + jit, p["Fuzzy"], s=12, alpha=0.35, color=e.AZUL, edgecolor="none")
    ax.plot([0, 20], [0, 20], color=e.TINTA, lw=1.2, ls="--")
    ax.axhline(9.5, color=e.CINZA, ls=":", lw=1)
    ax.axvline(9.5, color=e.CINZA, ls=":", lw=1)
    ax.set_xlim(-0.5, 20.5)
    ax.set_ylim(-0.5, 20.5)
```

```
    ax.set_xlabel("Nota real (G3)")
    ax.set_ylabel("Nota prevista pelo fuzzy")
    fig.tight_layout()
    st.pyplot(fig)
    e.como_ler("cada ponto é um aluno; quanto mais perto da linha tracejada, melhor a previsão. As linhas
pontilhadas marcam o corte 10.")
with col2:
    st.subheader("Curva ROC (detectar quem reprova)")
    fig, ax = plt.subplots(figsize=(4.6, 4.4))
    for nome, cor in [("Fuzzy", e.AZUL), ("Regressão linear", e.LARANJA), ("Floresta aleatória", e.VERDE)]:
        fpr, tpr, _ = roc_curve(df.reprovado, -p[nome])
        ax.plot(fpr, tpr, color=cor, lw=2.2 if nome == "Fuzzy" else 1.5, label=f"{nome} (AUC
{roc_auc_score(df.reprovado, -p[nome]):.2f})")
    ax.plot([0, 1], [0, 1], color=e.CINZA, ls="--", lw=1)
    ax.set_xlabel("Falsos alarmes (aprovados marcados em risco)")
    ax.set_ylabel("Reprovados detectados")
    ax.legend(loc="lower right", fontsize=8)
    fig.tight_layout()
    st.pyplot(fig)
    e.como_ler("quanto mais a curva sobe rápido para o canto superior esquerdo, melhor o modelo em separar quem
reprova de quem passa.")

# Alerta precoce: se o professor acompanhar só os N% de maior risco, quantos reprovados pega?
st.subheader("Alerta precoce: acompanhando só quem o fuzzy aponta")
risco = p["Fuzzy"].argsort(kind="stable")  # menor nota prevista = maior risco
ordem = df.reprovado.to_numpy()[risco]
fracoes = [0.1, 0.2, 0.3, 0.4]
linhas = []
for fr in fracoes:
    k = int(round(len(df) * fr))
    pegos = int(ordem[:k].sum())
    linhas.append({"Alunos acompanhados": f"{fr:.0%} ({k})", "Reprovados capturados": f"{pegos} de
{int(df.reprovado.sum())}", "Fração dos reprovados": pegos / df.reprovado.sum()})
st.dataframe(
    pd.DataFrame(linhas).style.format({"Fração dos reprovados": "{:.0%}"}),
    hide_index=True,
    width="stretch",
)
e.como_ler("se a escola acompanhar de perto apenas os alunos com a menor nota prevista, esta tabela mostra quanto
dos reprovados reais ela alcança.")

st.subheader("Conclusões")
if escolha == "A":
    st.markdown(
        f"""
- Sem nenhuma nota parcial, **prever o valor exato da nota é difícil para qualquer modelo** (o erro de todos fica
perto de 2 pontos, pouco abaixo do chute).
- Mas como **alerta de risco** o fuzzy funciona: AUC **{m_f['AUC (risco de reprovar)']:.2f}**, praticamente igual
à regressão linear ({m_l['AUC (risco de reprovar)']:.2f}),
  e com a vantagem de ser **totalmente explicável**: cada alerta vem com as regras que o causaram.
- Atenção: o "acerto aprovado/reprovado" com corte fixo em 10 fica abaixo do chute, porque só 15% reprovam. O
valor do fuzzy está em **ordenar o risco** (AUC e tabela de alerta precoce).
- O erro médio do fuzzy é um pouco maior que o da regressão, provavelmente porque sua saída é "agrupada" em poucos
conjuntos: ele ordena bem o risco, mas não calibra a nota fina.
"""
    )
else:
    st.markdown(
        f"""
- Com a nota do 2º período, o fuzzy passa a acertar a nota final com erro médio de cerca de **{m_f['Erro médio
(MAE)']:.1f} ponto**
  e AUC de **{m_f['AUC (risco de reprovar)']:.2f}**.
- A regressão linear ainda é um pouco mais precisa ({m_l['Erro médio (MAE)']:.2f} de erro), pois a nota final é
quase uma reta da nota do 2º período.
  O fuzzy fica perto e entrega o raciocínio por trás de cada previsão.
- As regras de correção (reprovação prévia, muito estudo) ajustam a previsão para cima ou para baixo, como faria
um professor experiente.
"""
    )
st.info(
    "**Limitação:** os conjuntos e regras foram definidos observando esta própria base; os números acima são, por
isso, "
    "um pouco otimistas para dados novos. Numa aplicação real, validaríamos com outra turma."
)
```

Testes `tests_fuzzy.py`

Conferência do motor contra o scikit-fuzzy e propriedades básicas.

```python
"""Testes do sistema fuzzy. Rode com:  python tests_fuzzy.py

1. Confere o motor NumPy (fuzzy_core) contra o motor de referência do scikit-fuzzy
   (skfuzzy.control), linha a linha, nos dois sistemas.
2. Confere propriedades básicas: pertinências entre 0 e 1, nota sempre dentro de 0–20,
   monotonicidade (mais G2 -> nota maior) e que nenhum aluno fica sem regra ativa.
"""
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

from fuzzy_core import SISTEMAS, carregar_dados, U_NOTA


def montar_skfuzzy(sistema):
    """Constrói o mesmo sistema com a API de controle do scikit-fuzzy."""
    ant = {}
    for n, v in sistema.entradas.items():
        lo, hi = v.universo
        # linspace garante que o último ponto do universo seja exatamente `hi`
        # (com arange, erro de ponto flutuante zera os conjuntos que terminam na borda).
        a = ctrl.Antecedent(np.linspace(lo, hi, int(round((hi - lo) / 0.01)) + 1), n)
        for c, (tipo, p) in v.conjuntos.items():
            a[c] = fuzz.trimf(a.universe, p) if tipo == "tri" else fuzz.trapmf(a.universe, p)
        ant[n] = a
    out = ctrl.Consequent(U_NOTA, "G3", defuzzify_method="centroid")
    for c, (tipo, p) in sistema.saida.conjuntos.items():
        out[c] = fuzz.trimf(U_NOTA, p) if tipo == "tri" else fuzz.trapmf(U_NOTA, p)
    regras = []
    for r in sistema.regras:
        cond = None
        for var, conj in r.se.items():
            t = ant[var][conj]
            cond = t if cond is None else cond & t
        regras.append(ctrl.Rule(cond, out[r.entao]))
    return ctrl.ControlSystem(regras)


def testar_contra_skfuzzy(chave, n=40):
    sis = SISTEMAS[chave]()
    df = carregar_dados().sample(n, random_state=7)
    nossa = sis.prever(df)
    cs = montar_skfuzzy(sis)
    ref = []
    for _, linha in df.iterrows():
        sim = ctrl.ControlSystemSimulation(cs)
        for nome in sis.entradas:
            sim.input[nome] = float(linha[nome])
        sim.compute()
        ref.append(sim.output["G3"])
    dif = np.max(np.abs(nossa - np.array(ref)))
    assert dif < 0.05, f"Sistema {chave}: diferença máxima {dif:.4f} contra skfuzzy"
    return dif


def testar_propriedades(chave):
    sis = SISTEMAS[chave]()
    df = carregar_dados()
    for n, v in sis.entradas.items():
        for c in v.conjuntos:
            mu = v.pertinencia(df[n].to_numpy(), c)
            assert mu.min() >= 0 and mu.max() <= 1, f"pertinência fora de [0,1] em {n}/{c}"
    f = sis.forcas(df)
    assert (f.sum(axis=1) > 0).all(), "há aluno sem nenhuma regra ativa"
    p = sis.prever(df)
    assert p.min() >= 0 and p.max() <= 20, "nota fora de 0–20"
    return float(p.min()), float(p.max())


def testar_monotonia_b():
    sis = SISTEMAS["B"]()
    notas = [sis.prever_um(G2=g, failures=0, studytime=2)["nota"] for g in range(0, 21)]
    assert all(b >= a - 1e-9 for a, b in zip(notas, notas[1:])), "nota deveria subir com G2"


if __name__ == "__main__":
    for k in SISTEMAS:
        print(f"Sistema {k}: diferença máx. contra skfuzzy = {testar_contra_skfuzzy(k):.4f}")
        lo, hi = testar_propriedades(k)
        print(f"Sistema {k}: propriedades OK (previsões entre {lo:.1f} e {hi:.1f})")
```

```
    testar_monotonia_b()
    print("Monotonia do sistema B OK")
    print("TUDO OK")
```

4. Aplicação rodando (prints)

Capturas do app Streamlit executando `streamlit run principal.py`.

Página 1 · A base de dados

📊 Lógica Fuzzy aplicada ao desempenho de alunos

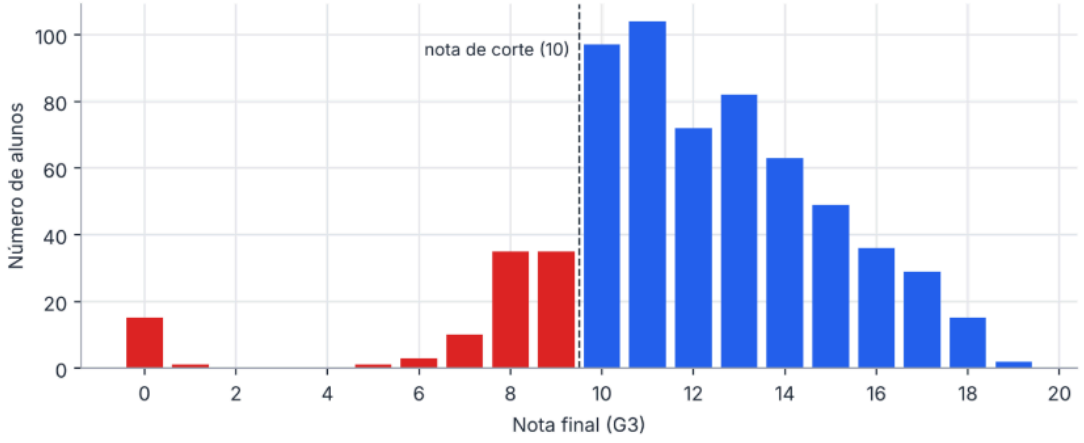
Objetivo: prever a **nota final (G3, de 0 a 20)** de alunos do ensino médio português usando **lógica fuzzy**, com regras que um professor entenderia e poderia conferir, sem "caixa-preta".

Por que fuzzy? Hábitos de estudo não têm fronteiras nítidas. Quem estuda "entre 2 e 5 horas" não é *pouco* nem *muito*, é um pouco dos dois. A lógica fuzzy trabalha com esses **graus** (0 a 1) e permite escrever regras em linguagem comum, como *"SE estuda muito E nunca reprovou ENTÃO a nota tende a ser boa"*.

› 📖 Glossário rápido (clique para abrir)

Alunos	Nota final média	Reprovados (G3 < 10)	Valores ausentes
649	11.9	100 (15%)	0

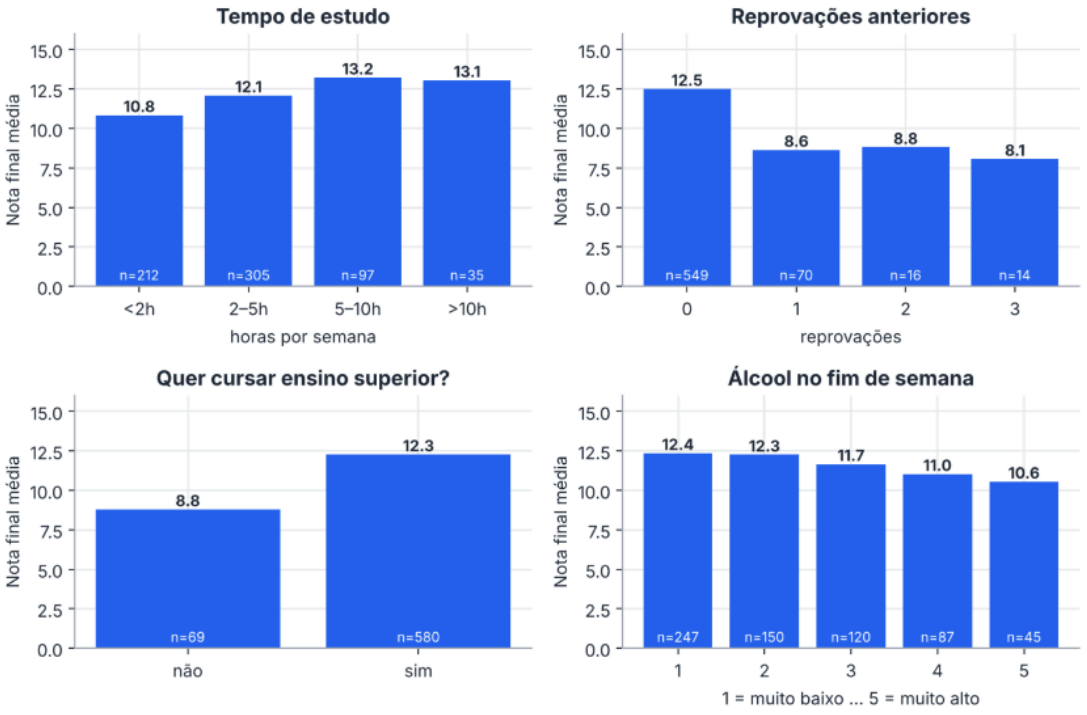
Como as notas se distribuem



Como ler: cada barra é uma nota; as vermelhas são reprovações (abaixo de 10). A maioria fica entre 10 e 14, e há 15 alunos com nota zero (provavelmente desistências), uma cauda difícil de prever.

O que muda a nota? (antes de qualquer modelo)

Página 1 · A base de dados (parte 1)



Como ler: cada barra é a nota média de um grupo (o *n* mostra quantos alunos há nele). Quem já reprovou e quem não pretende fazer faculdade tem nota bem menor; quem estuda mais e bebe menos tende a ir melhor.

Principais achados

- **Reprovações pesam muito:** quem nunca reprovou tem média **12.5**; quem reprovou uma vez, **8.6**.
- **Ambição importa:** quem quer ensino superior tem média **12.3**, contra **8.8** de quem não quer.
- **Estudo ajuda, com limite:** a média sobe de **10.8** (<2h) para **13.2** (5–10h) e estabiliza depois.
- **As notas parciais dominam:** a correlação de G2 com a nota final é **0.92** (G1: 0.83). Por isso montamos **dois sistemas**: o **A** usa só o perfil do aluno (serve como alerta precoce, antes de qualquer prova) e o **B** acrescenta a nota do 2º período.
- **Faltas quase não explicam a nota** nesta base (correlação de -0.09), por isso ficaram fora das regras.

› Ver uma amostra dos dados

Página 1 · A base de dados (parte 2)

Página 2 · Conjuntos fuzzy

📈 Conjuntos fuzzy: do número ao grau

Aqui cada variável vira **conjuntos fuzzy**. Em vez de "estuda pouco: sim/não", cada valor recebe um **grau de pertinência** de 0 a 1 em cada conjunto. As curvas abaixo (**funções de pertinência**) são triângulos e trapézios, desenhados a partir do que os dados mostram (por exemplo, onde a nota média muda de faixa). **Mova o controle** para ver a fuzzificação de um valor.

Sistema

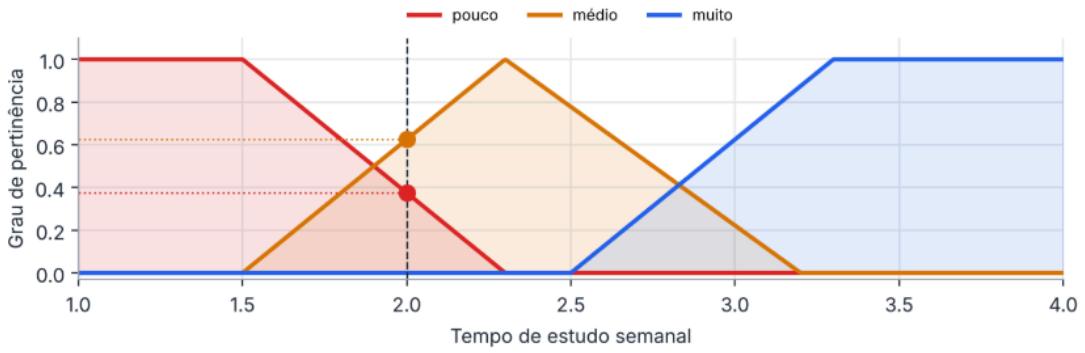
(•) A · Perfil do aluno (sem notas parciais) ( ) B · Com a nota do 2º período (G2)

Tempo de estudo semanal

Escala: 1 = <2h · 2 = 2–5h · 3 = 5–10h · 4 = >10h

Valor para fuzzificar

2



Valor **2** pertence a: **pouco** = 0.37 · **médio** = 0.63

Como ler: cada curva é um conjunto; a altura mostra o grau (1 = pertence totalmente, 0 = não pertence). A linha tracejada é o valor escolhido e os pontos mostram os graus que ele recebe. Onde as curvas se cruzam, o valor é um pouco de cada conjunto, e é aí que a lógica fuzzy se diferencia da lógica clássica.

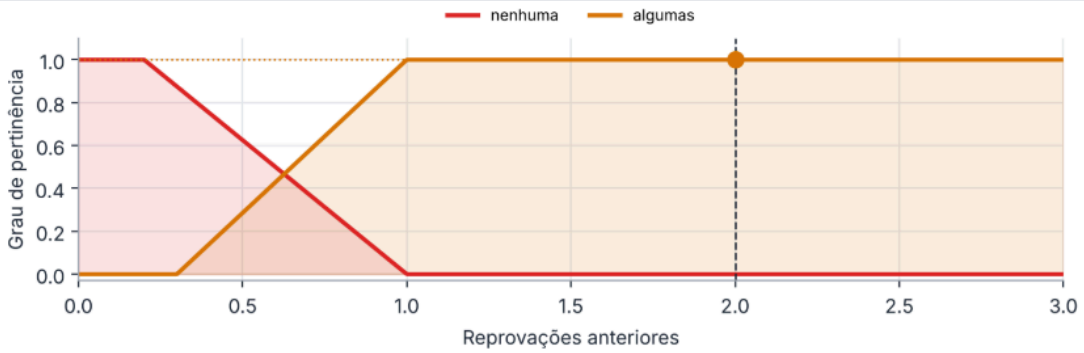
Reprovações anteriores

Escala: quantidade

Valor para fuzzificar

2

Página 2 · Conjuntos fuzzy (parte 1)



Valor **2** pertence a: **algumas** = 1.00

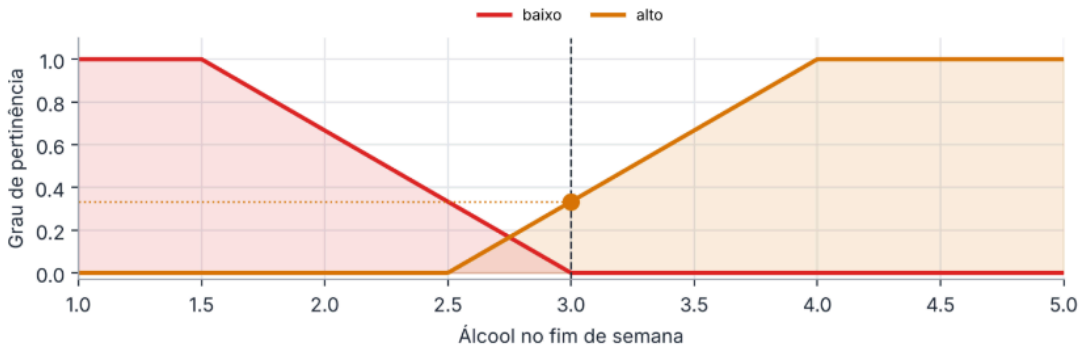
Como ler: cada curva é um conjunto; a altura mostra o grau (1 = pertence totalmente, 0 = não pertence). A linha tracejada é o valor escolhido e os pontos mostram os graus que ele recebe. Onde as curvas se cruzam, o valor é um pouco de cada conjunto, e é aí que a lógica fuzzy se diferencia da lógica clássica.

Álcool no fim de semana

Escala: 1 = muito baixo … 5 = muito alto

Valor para fuzzificar





Valor **3** pertence a: **alto** = 0.33

Como ler: cada curva é um conjunto; a altura mostra o grau (1 = pertence totalmente, 0 = não pertence). A linha tracejada é o valor escolhido e os pontos mostram os graus que ele recebe. Onde as curvas se cruzam, o valor é um pouco de cada conjunto, e é aí que a lógica fuzzy se diferencia da lógica clássica.

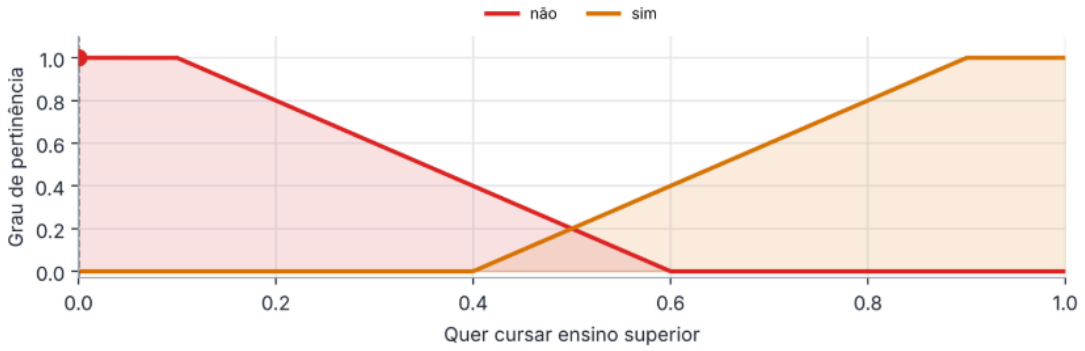
Quer cursar ensino superior

Escala: 0 = não · 1 = sim

Página 2 · Conjuntos fuzzy (parte 2)

Valor para fuzzificar

0



Valor **0** pertence a: **não** = 1.00

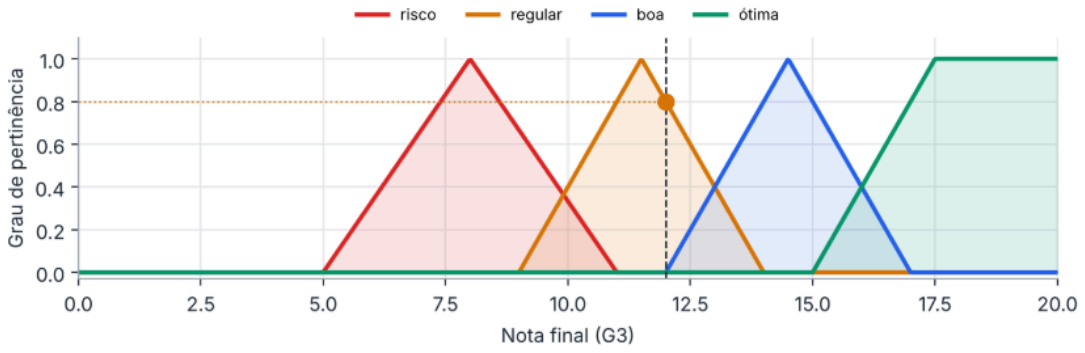
Como ler: cada curva é um conjunto; a altura mostra o grau (1 = pertence totalmente, 0 = não pertence). A linha tracejada é o valor escolhido e os pontos mostram os graus que ele recebe. Onde as curvas se cruzam, o valor é um pouco de cada conjunto, e é aí que a lógica fuzzy se diferencia da lógica clássica.

🎯 Saída: Nota final (G3)

Escala: pontos

Valor para fuzzificar

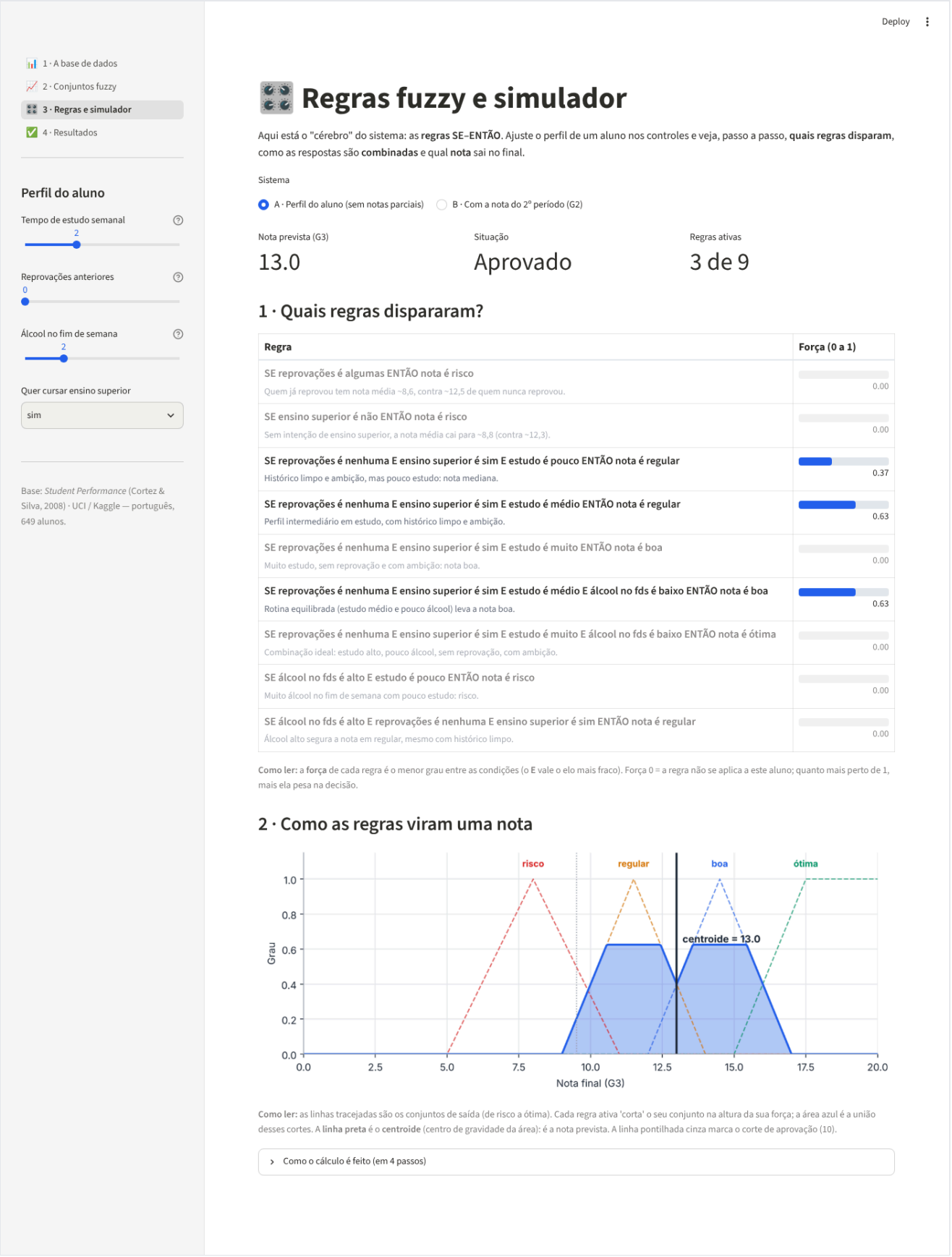
12.00



Valor **12** pertence a: **regular** = 0.80

Como ler: cada curva é um conjunto; a altura mostra o grau (1 = pertence totalmente, 0 = não pertence). A linha tracejada é o valor escolhido e os pontos mostram os graus que ele recebe. Onde as curvas se cruzam, o valor é um pouco de cada conjunto, e é aí que a lógica fuzzy se diferencia da lógica clássica.

Página 3 · Regras e simulador



Sistema A: perfil com estudo 2, sem reprovações, álcool 2, quer ensino superior. As barras mostram a força de cada regra.

Deploy

1 · A base de dados
2 · Conjuntos fuzzy
3 · Regras e simulador
4 · Resultados

Perfil do aluno

Nota do 2º período (G2) 11.00

Reprovações anteriores 0

Tempo de estudo semanal 2

Base: *Student Performance* (Cortez & Silva, 2008) · UCI / Kaggle — português, 649 alunos.

Regras fuzzy e simulador

Aqui está o "cérebro" do sistema: as **regras SE–ENTÃO**. Ajuste o perfil de um aluno nos controles e veja, passo a passo, **quais regras disparam**, como as respostas são **combinadas** e qual **nota** sai no final.

Sistema

A · Perfil do aluno (sem notas parciais) B · Com a nota do 2º período (G2)

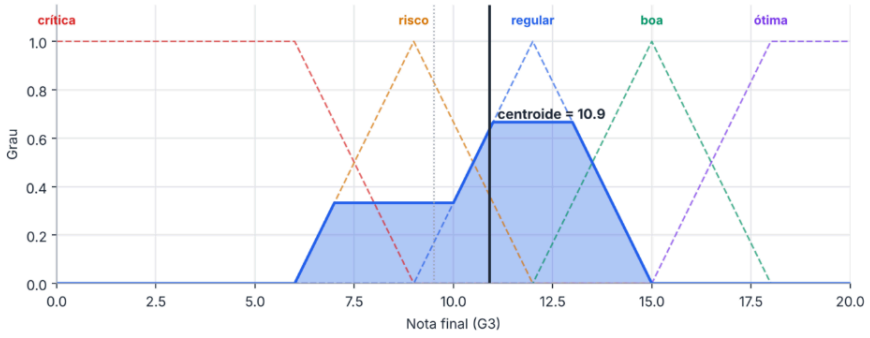
Nota prevista (G3)	Situação	Regras ativas
10.9	Aprovado	2 de 9

1 · Quais regras dispararam?

Regra	Força (0 a 1)
SE nota G2 é muito baixa ENTÃO nota é crítica Nota muito baixa no 2º período costuma terminar crítica.	0.00
SE nota G2 é baixa ENTÃO nota é risco Nota baixa no 2º período tende a continuar baixa.	0.33
SE nota G2 é média ENTÃO nota é regular Nota média se mantém média.	0.67
SE nota G2 é boa ENTÃO nota é boa Nota boa se mantém boa.	0.00
SE nota G2 é alta ENTÃO nota é ótima Nota alta se mantém ótima.	0.00
SE nota G2 é média E reprovações é algumas ENTÃO nota é risco Nota média com reprovação prévia tende a cair para a faixa de risco.	0.00
SE nota G2 é boa E reprovações é algumas ENTÃO nota é regular Nota boa com reprovação prévia tende a recuar um degrau.	0.00
SE nota G2 é baixa E estudo é muito ENTÃO nota é regular Nota baixa, mas com muito estudo: tende a recuperar.	0.00
SE nota G2 é média E estudo é muito ENTÃO nota é boa Nota média com muito estudo: tende a subir.	0.00

Como ler: a **força** de cada regra é o menor grau entre as condições (o **E** vale o elo mais fraco). Força 0 = a regra não se aplica a este aluno; quanto mais perto de 1, mais ela pesa na decisão.

2 · Como as regras viram uma nota



Como ler: as linhas tracejadas são os conjuntos de saída (de risco a ótima). Cada regra ativa 'corta' o seu conjunto na altura da sua força; a área azul é a união desses cortes. A **linha preta** é o **centroide** (centro de gravidade da área): é a nota prevista. A linha pontilhada cinza marca o corte de aprovação (10).

› Como o cálculo é feito (em 4 passos)

Sistema B: com G2 = 11, duas regras disparam (nota baixa e nota média) e o centroide cai entre elas.

Página 4 · Resultados

✅ Resultados: o fuzzy acerta?

Comparamos os dois sistemas fuzzy com **três referências**: um chute (sempre prever a média), uma **regressão linear** e uma **floresta aleatória** (as duas com validação cruzada de 5 partes, para ser justo). As regras fuzzy foram escritas à mão a partir do que a página 1 mostrou; **não há treino automático**.

Sistema

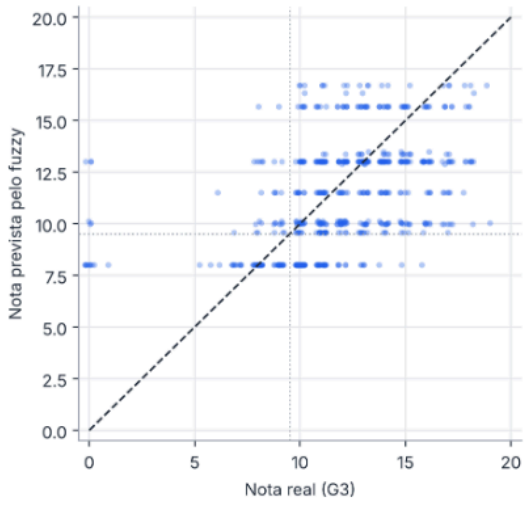
(●) A · Perfil do aluno (sem notas parciais) ( ) B · Com a nota do 2º período (G2)

Erro médio do fuzzy	AUC do fuzzy	R^2 do fuzzy
2.35 pts	0.80	0.06
↑ +0.28 vs regressão	↓ -0.00 vs regressão	

Modelo	Erro médio (MAE)	R^2	AUC (risco de reprovar)
Fuzzy	2.35	0.06	0.80
Chute (média)	2.41	0.00	0.50
Regressão linear	2.08	0.23	0.80
Floresta aleatória	2.08	0.24	0.79

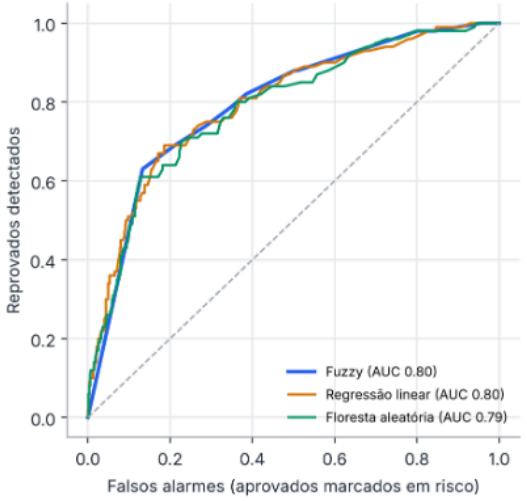
Como ler: MAE = quantos pontos de nota o modelo erra em média (menor é melhor). **AUC** mede se o modelo põe os alunos que realmente reprovaram acima dos demais na lista de risco (0,5 = chute, 1 = perfeito). **R^2** = fração da variação das notas explicada.

Real × previsto



Como ler: cada ponto é um aluno; quanto mais perto da linha tracejada, melhor a previsão. As linhas pontilhadas marcam o corte 10.

Curva ROC (detectar quem reprova)



Como ler: quanto mais a curva sobe rápido para o canto superior esquerdo, melhor o modelo em separar quem reprova de quem passa.

Página 4 · Resultados (parte 1)

Alerta precoce: acompanhando só quem o fuzzy aponta

Alunos acompanhados	Reprovados capturados	Fração dos reprovados
10% (65)	21 de 100	21%
20% (130)	59 de 100	59%
30% (195)	69 de 100	69%
40% (260)	74 de 100	74%

Como ler: se a escola acompanhar de perto apenas os alunos com a menor nota prevista, esta tabela mostra quanto dos reprovados reais ela alcança.

Conclusões

- Sem nenhuma nota parcial, **prever o valor exato da nota é difícil para qualquer modelo** (o erro de todos fica perto de 2 pontos, pouco abaixo do chute).
- Mas como **alerta de risco** o fuzzy funciona: AUC **0.80**, praticamente igual à regressão linear (0.80), e com a vantagem de ser **totalmente explicável**: cada alerta vem com as regras que o causaram.
- Atenção: o "acerto aprovado/reprovado" com corte fixo em 10 fica abaixo do chute, porque só 15% reprovam. O valor do fuzzy está em **ordenar o risco** (AUC e tabela de alerta precoce).
- O erro médio do fuzzy é um pouco maior que o da regressão, provavelmente porque sua saída é "agrupada" em poucos conjuntos: ele ordena bem o risco, mas não calibra a nota fina.

Limitação: os conjuntos e regras foram definidos observando esta própria base; os números acima são, por isso, um pouco otimistas para dados novos. Numa aplicação real, validaríamos com outra turma.

Página 4 · Resultados (parte 2)

5. Resultados e conclusões

Comparamos cada sistema com um chute (prever sempre a média), uma regressão linear e uma floresta aleatória (as duas com validação cruzada de 5 partes, sobre as mesmas variáveis). **AUC** mede a capacidade de colocar os alunos que reprovaram no topo da lista de risco (0,5 = chute; 1 = perfeito).

Sistema A · perfil do aluno (sem notas parciais)

Modelo	Erro médio (MAE)	R^2	AUC (risco)	Acerto aprov./reprov.
Fuzzy	**2.35**	**0.06**	**0.80**	**72%**
Chute (média)	2.41	0.00	0.50	85%
Regressão linear	2.08	0.23	0.80	86%
Floresta aleatória	2.08	0.24	0.79	83%

Sistema B · com a nota do 2º período

Modelo	Erro médio (MAE)	R^2	AUC (risco)	Acerto aprov./reprov.
Fuzzy	**0.98**	**0.79**	**0.96**	**90%**
Chute (média)	2.41	0.00	0.50	85%
Regressão linear	0.81	0.84	0.96	89%
Floresta aleatória	0.85	0.82	0.96	88%

Leitura dos resultados

- **Sem nota parcial, prever o valor exato é difícil para qualquer modelo:** o erro de todos fica perto de 2 pontos, pouco abaixo do chute (2.41). Como **alerta de risco**, porém, o fuzzy se sai bem: AUC de **0.80**, igual à regressão linear (0.80), e cada alerta vem com as regras que o causaram. Note que o "acerto aprovado/reprovado" do sistema A (72%) fica abaixo do chute (85%): como só 15% dos alunos reprovam, prever "aprovado" para todos já acerta muito; o valor do fuzzy está em **ordenar o risco** (AUC), não em fixar um corte único.
- **Alerta precoce (sistema A):** acompanhando só os 20% de alunos com menor nota prevista, a escola alcança **59 dos 100** reprovados; com 30%, **69**.
- **Com G2 (sistema B)** o fuzzy erra em média **0.98 ponto**, com AUC de **0.96** e 90% de acerto aprovado/reprovado. A regressão linear é um pouco mais precisa (0.81 de erro), pois a nota final é quase uma reta da nota do 2º período; o fuzzy fica perto e entrega o raciocínio.
- **Vantagem do fuzzy:** transparência. Qualquer previsão pode ser conferida regra a regra (página 3) e as regras podem ser corrigidas por um professor sem saber programar.

> **Limitações.** Os conjuntos e regras foram definidos observando esta própria base, então os números são um pouco otimistas para dados novos. O fuzzy ordena bem o risco, mas calibra pior a nota fina que uma regressão (saída agrupada em poucos conjuntos). A base inclui 15 alunos com nota zero (provável desistência), que nenhum modelo prevê bem. Como próximos passos: validar com outra turma e ajustar os conjuntos automaticamente (por exemplo, ANFIS).

Como reproduzir

`pip install -r requirements.txt` · `streamlit run principal.py` · `python tests_fuzzy.py`.
Repositório: `github.com/ooArthur/logica-fuzzy`. Dados: *Student Performance*, UCI Machine Learning Repository (P. Cortez e A. Silva, 2008).